

LAPORAN MINI PROJECT TASK MANAGER PRO

Aplikasi Manajemen Tugas Berbasis PySide6 dan SQLite
Tugas Matakuliah Pemrograman Visual



Oleh:

Nama: Raffi Fatthoni

NIM : F1D02310133

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MATARAM
2026**

A. Deskripsi Aplikasi dan Tujuan *Project*

Task Manager Pro adalah aplikasi manajemen tugas berbasis *desktop* yang dibangun menggunakan *framework* PySide6 (Python Qt *binding*) dengan penyimpanan data menggunakan SQLite. Aplikasi ini dirancang untuk membantu pengguna mengelola pekerjaan sehari-hari secara terstruktur melalui antarmuka grafis yang modern dan intuitif.

Aplikasi menerapkan sistem kategorisasi tugas berdasarkan prioritas (*High, Medium, Low*) dan status pengerjaan (*Todo, In Progress, Done*), dilengkapi dengan fitur pencarian *real-time* dan filter berbasis *pill buttons* untuk kemudahan navigasi.

1. Tujuan *Project*

- Membangun aplikasi GUI PySide6 yang fungsional dengan topik orisinal.
- Mengintegrasikan SQLite sebagai penyimpanan data yang persisten.
- Menerapkan *styling* visual menggunakan file QSS eksternal.
- Mengorganisir kode dengan prinsip *Separation of Concerns* (SoC).
- Mengimplementasikan mekanisme *signals* dan *slots* PySide6.

B. Metode Pengambilan Data

Project ini menerapkan prinsip *Separation of Concerns* (SoC) dengan membagi kode ke dalam empat *layer* yang jelas dan bertanggung jawab atas satu domain masing-masing.

1. Struktur Folder

<i>File / Folder</i>	Tanggung Jawab
main.py	<i>Entry point</i> . Memuat QSS dan menjalankan aplikasi.
styles/style.qss	<i>Layer Styling</i> : semua definisi visual QSS eksternal.
database/db_manager.py	<i>Layer Data</i> : koneksi SQLite, inisialisasi tabel, dan operasi CRUD.
controllers/task_controller.py	<i>Layer Logika</i> : validasi <i>input</i> dan menjembatani <i>View</i> ke Database.
ui/main_window.py	<i>Layer View</i> : jendela utama, tabel, <i>stat cards</i> , dan <i>toolbar</i> .
ui/task_dialog.py	<i>Layer View</i> : dialog form <i>Add/Edit task</i> terpisah dari jendela utama.

2. Penerapan SoC

Setiap *layer* hanya berkomunikasi dengan *layer* yang langsung berdekatan:

- View* hanya memanggil *method* Controller, tidak pernah langsung ke Database.
- Controller hanya memanggil *method* DatabaseManager, tidak tahu detail SQL.
- DatabaseManager hanya berinteraksi dengan SQLite, tidak tahu logika bisnis.

- d. *Styling* sepenuhnya di file QSS eksternal, tidak ada “`setStyleSheet()`” di dalam kode logic.

C. Desain Database

Database menggunakan SQLite dengan satu tabel utama bernama *tasks*. Database dibuat otomatis saat aplikasi pertama kali dijalankan menggunakan *query* CREATE TABLE IF NOT EXISTS.

1. Schema Tabel: tasks

Kolom	Type	Constraint	Keterangan
id	INTEGER	PK AUTOINCREMENT	<i>Primary key</i> , unik per <i>task</i> .
title	TEXT	NOT NULL	Judul <i>task</i> , wajib diisi.
category	TEXT	DEFAULT 'General'	Kategori: <i>General</i> , <i>Work</i> , <i>Study</i> , dll.
priority	TEXT	DEFAULT 'Medium'	Nilai: <i>High</i> , <i>Medium</i> , atau <i>Low</i> .
status	TEXT	DEFAULT 'Todo'	Nilai: <i>Todo</i> , <i>In Progress</i> , atau <i>Done</i> .
due_date	TEXT	NOT NULL	Tanggal tenggat, format: yyyy-MM-dd.
description	TEXT	DEFAULT ''	Deskripsi singkat, boleh kosong.
created_at	TEXT	NOT NULL	<i>Timestamp</i> dibuat, format: yyyy-MM-dd HH:mm:ss.

2. Operasi Database yang Diimplementasikan

Operasi	Method	SQL
<i>CREATE</i>	<code>create_task(data)</code>	INSERT INTO tasks VALUES (:field, ...)
<i>READ</i>	<code>get_all_tasks()</code> , <code>search_tasks()</code>	SELECT * FROM tasks ORDER BY priority, due_date
<i>UPDATE</i>	<code>update_task(id, data)</code>	UPDATE tasks SET ... WHERE id=:id
<i>DELETE</i>	<code>delete_task(id)</code> , <code>delete_all_done()</code>	DELETE FROM tasks WHERE id=? / status='Done'

D. Fitur Utama dan Cara Kerjanya

1. Tambah Task (Create)

Pengguna mengklik tombol Tambah di *toolbar*, yang memanggil *method* “`_add_task()`” di *MainWindow*. *Dialog* TaskDialog terbuka secara modal dengan 6 *field input*. Setelah *form* diisi dan tombol Simpan ditekan, data dikemas sebagai *dictionary* dan dikirim ke “`TaskController.add()`”. Controller menjalankan validasi, kemudian meneruskan ke “`DatabaseManager.create_task()`” yang mengeksekusi *query* INSERT SQL.

2. Edit Task (Update)

Pengguna memilih baris di tabel lalu mengklik tombol *Edit* (atau *double-click* baris). *Dialog* yang sama dibuka namun dalam mode edit, dengan *field* sudah terisi data *existing*. Setelah

disimpan, controller memanggil “`DatabaseManager.update_task()`” dengan *query* UPDATE SQL menggunakan WHERE id.

3. Hapus Task (Delete)

Pengguna memilih baris dan mengklik Hapus. QMessageBox konfirmasi muncul untuk mencegah penghapusan tidak sengaja. Jika dikonfirmasi, controller memanggil “`DatabaseManager.delete_task()`” dengan *query* DELETE SQL.

4. Filter dan Pencarian *Real-time*

Filter *pills* di *toolbar* memungkinkan pengguna memfilter by Status atau Prioritas dengan satu klik. Kolom pencarian menggunakan *signal* `textChanged` yang terhubung ke “`_on_search()`”. Keduanya memanggil “`DatabaseManager.search_tasks()`” dengan parameter WHERE yang dibangun secara dinamis.

5. Stat Cards

Lima kartu statistik di atas tabel menampilkan jumlah Total, *Done*, *In Progress*, *Todo*, dan *High Priority*. *Stat cards* diperbarui secara otomatis setiap kali data berubah melalui *method* “`_update_stats()`” yang memanggil “`DatabaseManager.get_stats()`”.

6. Deteksi *Overdue*

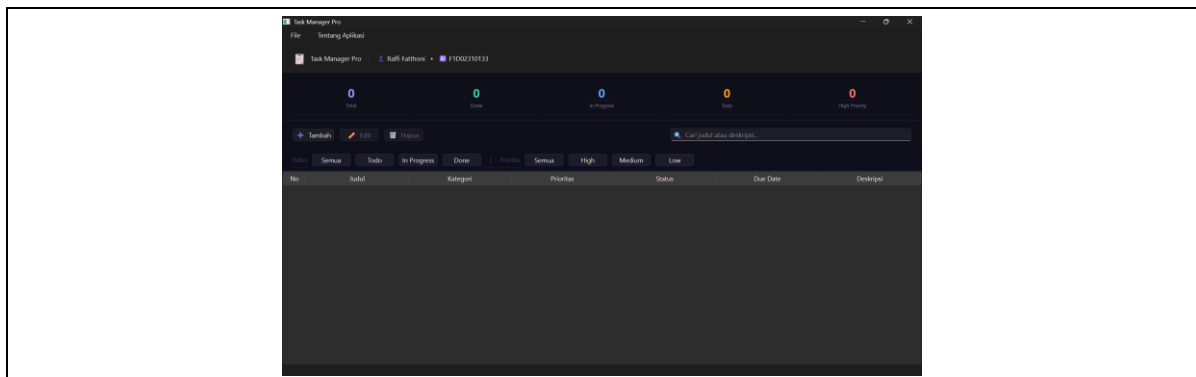
Saat mengisi tabel, *due date* setiap *task* dibandingkan dengan “`QDate.currentDate()`”. Jika tanggal sudah lewat, teks *due date* ditampilkan merah dengan ikon peringatan.

7. Styling QSS Eksternal

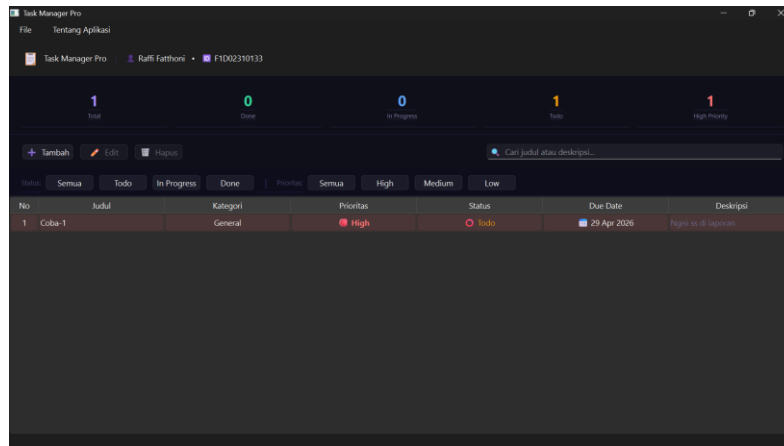
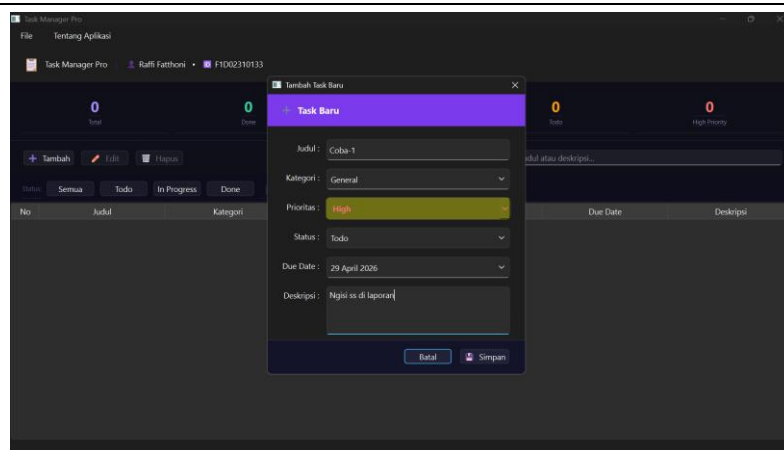
Seluruh *styling* visual dimuat dari *file* `styles/style.qss` di *method* “`load_stylesheet()`” pada `main.py` menggunakan “`app.setStyleSheet()`”. Tidak ada *hardcoded style* di dalam kode logika aplikasi.

E. Screenshot Antarmuka

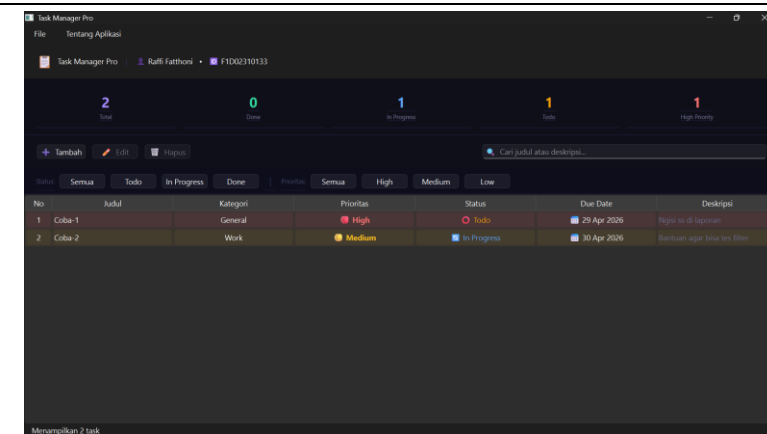
1. Tampilan jendela utama dengan beberapa *task*

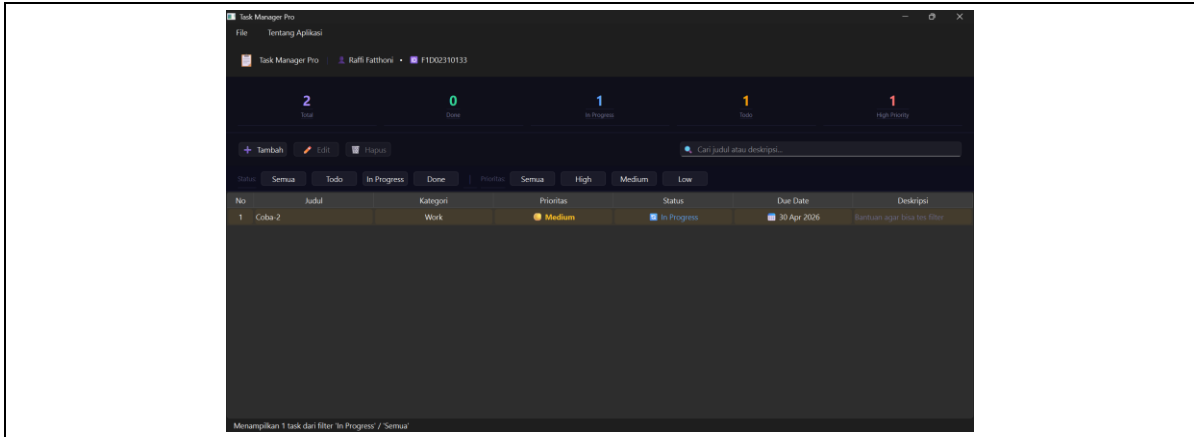


2. *Dialog form Tambah Task*

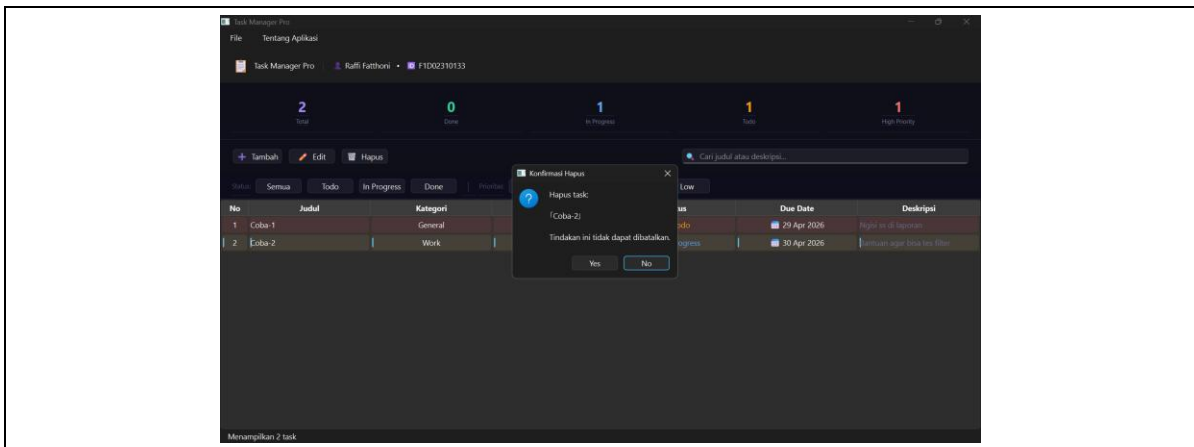


3. Tampilan filter *pills* aktif (misal filter status 'In Progress')

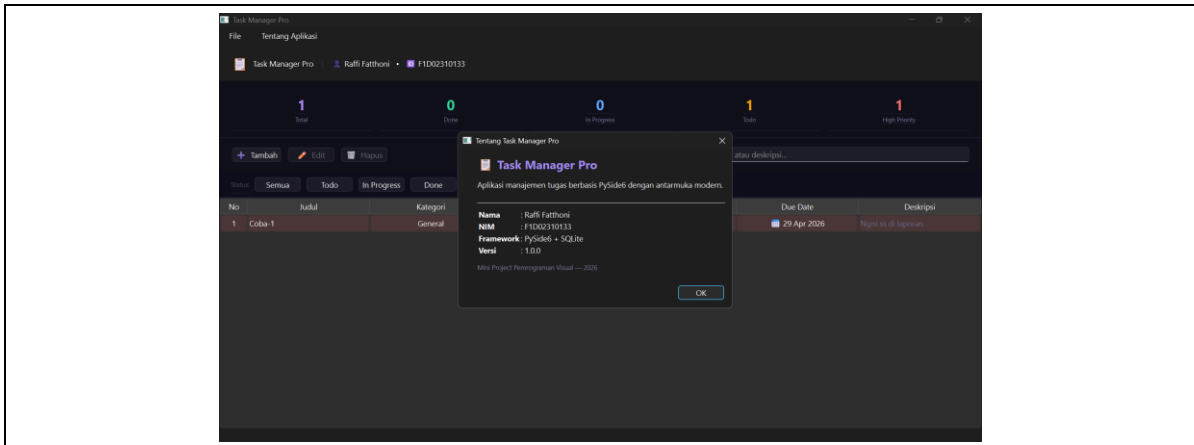




4. QMessageBox konfirmasi hapus *task*



5. Menu bar, Tentang Aplikasi



F. Link YouTube dan GitHub

1. Video Penjelasan Fitur CRUD

[]

2. Repository GitHub

[[Spicarf/mini-project-uts](https://github.com/Spicarf/mini-project-uts)]